

第19-20课 PDF解析整理版

来源：飞书「ChatBI 项目系统蓝图」解析整理版。原课件 PDF 附件无下载权限，本文为文档正文导出。

第 19-20课 PDF 解析整理版

第19-20课：Schema Linking + RAG 全链路融合实战合集

[要点] 课程总结与关系说明

两课核心关系：

第19课与第20课是第四阶段《让系统"理解业务"》的核心进阶与收官组合。

- 第19课（单点突破）：
聚焦于“指标知识模块”的RAG升级。解决了模型知道“查哪些表”但不知道“怎么算”的问题，通过语义检索替代关键词匹配，实现了指标计算公式和口径的精准注入。
- 第20课（全链路融合）：
作为阶段收官课，将第18课的Schema Linking（解决“查哪里”）与第19课的指标RAG（解决“怎么算”）统一整合进Text2SQL主链路，形成完整的业务语义理解层，并进行了全面的效果评估与系统升级。

学习路径建议：

必须先掌握第19课的指标RAG检索机制，才能理解第20课中双路检索系统的协作逻辑与集成方式。两课共同完成了从“规则驱动”到“语义驱动”的系统性跨越。

第19课：指标知识库设计与RAG检索实战

课程定位与目标

课程定位：

- 所处阶段：第四阶段《让系统"理解业务"——Schema Linking + RAG》第七节
- 本课核心问题：第18课的Schema Linking解决了"查哪些表和字段"，但模型仍然不知道"怎么算"——毛利率的公式是什么？费用率怎么计算？利润的定义包含哪些组成部分？本课用RAG（检索增强生成）从指标知识库中检索计算公式并注入Prompt。

本节课目标：

- 理解为什么需要升级指标知识模块（关键词匹配的局限性）

- 掌握指标元数据结构设计（分层、依赖链、口径说明）
- 用语义检索替代关键词匹配，实现 `indicator_retriever.py`
- 验证模糊表达的检索效果（如“各产品线赚了多少钱” → 产品线收入）

开场引入

回顾第9课的指标知识模块：

```
# 旧方案：纯关键词匹配
class IndicatorKnowledge:
    def detect_indicators(self, question):
        for alias in self.alias_map:
            if alias in question.lower():
                detected.append(...)
```

这个方案的核心逻辑是 `if alias in question` —— 只有当用户问题字面包含指标别名时才能命中。

[要点]

语义检索的解决思路：

把指标名、别名、定义、公式拼接成一段自然语言文本，用Embedding自动学习语义关系，新增指标时只需要写好定义，无需扩展别名规则。

一、指标元数据结构设计

从5条到13条

第9课的 `indicators.json` 只有5个指标、每个指标6个字段。本课扩展为：

- 13个核心指标：收入、成本、毛利、毛利率、期间费用、利润、客单价、订单量、研发费用率、销售费用率、产品线收入、区域收入、销量
- 每个指标11个字段：新增 `level`（分层）、`sql_template`（参考SQL）、`notes`（注意事项）

指标分层

完整元数据结构

```
{
  "name": "毛利率",
  "level": "派生指标",
  "aliases": ["毛利润率", "gross_margin", "产品利润率"],
  "definition": "毛利占收入的百分比，反映产品盈利效率。",
  "formula": "(毛利/收入) * 100",
  "sql_template": "SELECT (SUM(net_amount * rate_to_cny) - SUM((material_cost + labor_cost) * quantity)) / SUM(net_amount * rate_to_cny) * 100 ...",
```

```

"data_source": "sales_orders JOIN dim_products JOIN exchange_rates",
"depends_on": ["毛利", "收入"],
"time_field": "o.order_date",
"filters": ["o.order_status='completed'"],
"notes": "结果为百分比数值, 如 35.2 表示毛利率 35.2%"
}

```

每个字段的用途

二、向量化存储与RAG检索

检索文档构造

每个指标生成一段“检索文档”，被向量化后用于语义匹配：

```

search_text = (
    f"指标: {name} ({aliases_str}) \n"
    f"定义: {ind['definition']} \n"
    f"计算公式: {ind['formula']} "
)

```

设计意图：

- 指标名 + 别名：直接匹配包含指标名的问题
- 业务定义：捕捉语义等价表达（“赚了多少钱” → 与“收入”的定义语义接近）
- 计算公式：让包含字段名或计算逻辑的问题也能命中

ChromaDB存储

复用第15课模式，新建 `indicators` `collection`。注意：必须显式指定 `cosine` 距离，否则Chroma默认L2会导致相似度分数越界并触发警告。

```

CHROMA_PERSIST_DIR = os.path.join(os.path.dirname(__file__), "chroma_db", "indicators")

def _cosine_relevance_score_fn(distance: float) -> float:
    """将 Chroma 的 cosine 距离 [0, 2] 归一化为 [0, 1] 相似度得分。"""
    return 1.0 - distance / 2.0

def get_vectorstore() -> Chroma:
    return Chroma(
        collection_name="indicator_definitions",
        embedding_function=get_embeddings(),
        persist_directory=CHROMA_PERSIST_DIR,
        collection_metadata={"hnsw:space": "cosine"},
        relevance_score_fn=_cosine_relevance_score_fn,
    )

```

现在 `chroma_db/` 目录下有三个`collection`：

```
chroma_db/
├── tables/      # 第15课：表描述向量
├── fields/      # 第16课：字段描述向量
└── indicators/  # 第19课：指标定义向量
```

检索逻辑

```
def retrieve_indicators(query, top_k=3, score_threshold=0.3, expand_dependencies=True):
    # 1. 向量检索
    results = vectorstore.similarity_search_with_relevance_scores(query, k=top_k)

    # 2. 过滤低分
    matched = [r for r in results if score >= score_threshold]

    # 3. 依赖展开（关键！）
    if expand_dependencies:
        for item in matched:
            for dep_name in item["depends_on"]:
                if dep_name not in matched_names:
                    # 自动补充依赖指标的定义
                    ...
```

依赖指标自动展开

这是本课的核心设计——当检索到"利润"时，模型还需要知道"毛利"和"期间费用"的计算方式：

```
用户问："查询今年的利润"
检索命中：利润 (score: 0.72)
depends_on: ["毛利", "期间费用"]
自动展开：
- 毛利（依赖展开）→ 公式：收入 - 成本
- 期间费用（依赖展开）→ 公式：SUM(rd + selling + admin + finance)
```

注入Prompt后，模型不仅知道"利润 = 毛利 - 费用"，还知道"毛利"和"费用"分别怎么算。

[要点] 依赖展开的深度控制：

当前只展开一层（直接依赖）。如果"毛利"又依赖"收入"和"成本"，是否继续展开？本课控制为一层，避免注入过多信息。第20课全链路融合时可根据需要调整。

三、Prompt知识块生成

输出格式

```
def build_indicator_knowledge_block(query):
    indicators = retrieve_indicators(query)
    # 生成格式：
    """

    【指标知识】
    指标：毛利率
    定义：毛利占收入的百分比，反映产品盈利效率。
```

计算公式：(毛利/收入) * 100

数据来源：sales_orders JOIN dim_products JOIN exchange_rates

注意：结果为百分比数值，如 35.2 表示毛利率 35.2%

指标：毛利（依赖指标）

定义：销售收入减去销售成本。反映产品本身的盈利能力。

计算公式：SUM(net_amount * rate_to_cny) - SUM((material_cost + labor_cost) * quantity)

...

指标：收入（依赖指标）

定义：不含税销售收入，财务口径的销售额。

计算公式：SUM(o.net_amount * r.rate_to_cny)

...

"""

与prompt_builder的集成

旧方式（第9课关键词匹配）

```
from indicator_knowledge import IndicatorKnowledge
ik = IndicatorKnowledge()
knowledge_block = ik.build_knowledge_block(question)
```

新方式（第19课语义检索）

```
from indicator_retriever import build_indicator_knowledge_block
knowledge_block = build_indicator_knowledge_block(question)
```

接口完全兼容——返回值都是 str，直接传给 build_prompt(indicator_knowledge=...)。

四、完整代码运行

```
uv run indicator_retriever.py
```

输出示例（实际运行结果，分数随Embedding模型和API会有波动）：

```
=====
指标知识库 RAG 检索演示
=====
```

—— 构建指标向量索引 ——

指标索引构建完成：13 个指标已写入 ChromaDB

持久化路径：.../chroma_db/indicators

—— 语义检索测试 ——

```
=====
问题：查询上个月的毛利
```

说明：直接包含指标名 '毛利'

```
=====
毛利（相似度：0.477）[派生指标]
```

公式：SUM(o.net_amount * r.rate_to_cny) - SUM((p.material_cost + p.labor_cost) * o.quantity)

毛利率（相似度：0.426）[派生指标]

公式：(毛利/收入) * 100

利润（相似度：0.419）[复合指标]

公式：毛利 - 期间费用

收入（相似度：0.000）[原子指标] [依赖展开]

公式：SUM(o.net_amount * r.rate_to_cny)

销售成本（相似度：0.000）[原子指标] [依赖展开]

公式：SUM((p.material_cost + p.labor_cost) * o.quantity)

期间费用（相似度：0.000）[原子指标] [依赖展开]

公式：SUM(rd_expense + selling_expense + admin_expense + finance_expense)

问题：各产品线赚了多少钱

说明：不含 '收入' 但语义等价 '产品线收入'

产品线收入（相似度：0.509）[派生指标]

公式：SUM(o.net_amount * r.rate_to_cny) GROUP BY p.product_line

收入（相似度：0.327）[原子指标]

公式：SUM(o.net_amount * r.rate_to_cny)

销量（相似度：0.319）[原子指标]

公式：SUM(quantity) WHERE order_status='completed'

问题：今年的毛利率是多少

说明：毛利率 → 依赖展开 '收入' 和 '毛利'

毛利率（相似度：0.539）[派生指标]

公式：(毛利/收入) * 100

毛利（相似度：0.483）[派生指标]

公式：SUM(o.net_amount * r.rate_to_cny) - SUM((p.material_cost + p.labor_cost) * o.quantity)

销售费用率（相似度：0.352）[派生指标]

公式：SUM(selling_expense)/收入 * 100

收入（相似度：0.000）[原子指标] [依赖展开]

公式：SUM(o.net_amount * r.rate_to_cny)

销售成本（相似度：0.000）[原子指标] [依赖展开]

公式：SUM((p.material_cost + p.labor_cost) * o.quantity)

问题：今天天气怎么样

说明：无关问题，不应命中任何指标

（未命中任何指标）

关键词匹配 vs 语义检索对比

问题：各产品线赚了多少钱

关键词匹配：未命中

语义检索：['产品线收入', '收入', '销量']

问题：卖出去的东西成本多少

关键词匹配：['销售成本']

语义检索：['销售成本', '销售费用率', '毛利']

问题：上个月的利润

关键词匹配：['利润']

语义检索：['利润', '毛利', '产品线收入']

五、关键词匹配vs语义检索对比

测试案例

关键词匹配的覆盖率约40-50%，语义检索可达到80%+。语义检索不是"字面包含"，而是捕捉语义关联，因此也会召回一些相关但非目标的指标（如"利润"查询中同时出现"产品线收入"），这属于正常扩散，可通过top_k和阈值控制。

性能对比

语义检索增加的50-100ms延迟在实际使用中可以忽略（用户不会感知到），但覆盖率的提升是质变。

小结

本节课完成了指标知识模块的RAG升级：

核心改进：

- 关键词匹配 → 语义检索（覆盖率40% → 80%+）
- 5个指标 → 13个核心指标（完整覆盖分析场景）
- 新增依赖展开机制（自动补充上游指标定义）
- 指标分层设计（原子/派生/复合）为复杂查询提供结构化指引

下一课将把Schema Linking和指标RAG两套系统统一整合到Text2SQL链路中。

附录：完整代码

完整代码文件路径：

- code/chatbi_mvp/indicator_retriever.py（检索模块）
- code/chatbi_mvp/indicators_full.json（完整指标知识库）

第20课：Schema Linking + RAG 全链路融合与系统升级

课程定位与目标

课程定位：

- 所处阶段：第四阶段《让系统"理解业务"——Schema Linking + RAG》第八节（阶段收官课）
- 本课核心问题：将Schema

Linking（表/字段/Join动态召回）和指标RAG（语义检索计算公式）两套检索系统统一整合到Text2SQL链路中，形成完整的“业务语义理解层”。

本节课目标：

- 将Schema Linking + 指标RAG集成到 main.py / ChatBISystem
- 理解两套检索系统的协作关系：Schema Linking提供“查哪里”，指标RAG提供“怎么算”
- 全面效果评估：基线 vs 当前系统
- 识别遗留问题，为第五阶段做铺垫

开场引入

回顾第四阶段构建的模块全景：

本课的任务：把这两条并行的检索线路接入 main.py，让 ChatBISystem.run() 一个函数调用完成全部工

一、全链路架构

最终链路

两条检索路径互相独立、同时执行：Schema Linking回答“查哪里”，指标RAG回答“怎么算”。

两套系统的协作

它们互补而非重叠——Schema Linking不知道“毛利率怎么算”，指标RAG不知道“应该Join哪些表”。

二、集成到ChatBISystem

新增参数

```
class ChatBISystem:
    def run(
        self,
        user_question: str,
        use_few_shot: bool = True,
        use_rules: bool = True,
        use_guards: bool = True,
        use_indicator_knowledge: bool = True, # 旧版关键词匹配（第9课）
        use_schema_linking: bool = False,   # Schema Linking（第18课）← 新增
        use_indicator_rag: bool = False,    # 指标 RAG（第19课）← 新增
    ) -> dict:
```

设计原则

1. 默认关闭： 新功能默认False，不影响已有行为
2. 独立开关： Schema Linking和指标RAG可以分别启用
3. 优雅降级： 任一模块异常都不影响整体运行
4. RAG优先于关键词： 当 use_indicator_rag=True 时，优先用语义检索；失败时回退到关键词匹配

核心改造逻辑

```
def _resolve_indicator_context(
    self,
    user_question: str,
    use_indicator_knowledge: bool,
    use_indicator_rag: bool,
) -> tuple[list[str], str]:
    detected_indicators = []
    indicator_block = ""

    if use_indicator_rag:
        try:
            from indicator_retriever import retrieve_indicator_context
            context = retrieve_indicator_context(user_question)
            detected_indicators = context["detected_indicators"]
            indicator_block = context["indicator_block"]
        except Exception:
            detected_indicators = []
            indicator_block = ""

    if use_indicator_knowledge and not indicator_block:
        context = self.indicator_knowledge.get_indicator_context(user_question)
        detected_indicators = context["detected_indicators"]
        indicator_block = context["indicator_block"]
    elif use_indicator_knowledge:
        context = self.indicator_knowledge.get_indicator_context(user_question)
        detected_indicators = context["detected_indicators"]
        indicator_block = context["indicator_block"]

    return detected_indicators, indicator_block
```

```
detected_indicators, indicator_block = self._resolve_indicator_context(
    user_question=user_question,
    use_indicator_knowledge=use_indicator_knowledge,
    use_indicator_rag=use_indicator_rag,
)
```

Prompt 构造 (Schema Linking 在 build_prompt 内部处理)

```
system_msg, prompt = build_prompt(
    user_question,
    use_few_shot=use_few_shot,
    use_rules=use_rules,
    use_guards=use_guards,
    indicator_knowledge=indicator_block,
    use_schema_linking=use_schema_linking, # ← 传递开关
)
```

```
_resolve_indicator_context()
```

的作用，是把指标上下文解析收敛为一次调用、一次返回。这样无论使用指标RAG还是关键词匹配，都只需要拿到同一份结果：

- detected_indicators
- indicator_block

同步链路与流式链路都直接复用这一套指标上下文，主链路结构也会更稳定。

使用方式

```
system = ChatBISystem()

# 全能模式 (Schema Linking + 指标 RAG + 规则 + Few-shot)
result = system.run(
    "各产品线的毛利率",
    use_rules=True,
    use_guards=True,
    use_schema_linking=True,
    use_indicator_rag=True,
)

# 基线模式 (全量 Schema + 关键词匹配, 用于对比)
baseline = system.run(
    "各产品线的毛利率",
    use_rules=True,
    use_guards=True,
    use_schema_linking=False,
    use_indicator_rag=False,
    use_indicator_knowledge=True,
)
```

三、全面效果评估

评估框架

使用第8课的评估方法，对比三种配置：

Token消耗对比

Schema Linking大幅减少了Schema部分的Token，即使加上指标RAG的注入，总Token仍然降低了近一半。

SQL准确率预期

响应延迟分析

向量检索总增加~200ms，相对于LLM的1-3秒可以忽略。且Token减少反而可能加速LLM推理（输入更短生成更快）。 →

四、优化策略

以上三种策略均为生产环境进阶方向，可根据实际情况分别进行尝试。

五、遗留问题与第五阶段预告

仍然无法解决的问题类型

这些问题的共同特征：不是一条SQL能回答的。需要系统具备"拆解任务、分步执行、汇总结果"的能力。

第五阶段方向

第五阶段将引入Plan-and-Execute Agent：

1. 用户问题 → LLM生成执行计划（多个子步骤）
2. 每个子步骤独立执行（可能是SQL，也可能是其他操作）
3. 汇总所有子步骤的结果，生成最终回答

这超出了Text2SQL的范畴，进入了Agent系统的领域。

小结

本节课完成了第四阶段的收官——将Schema Linking + 指标RAG全链路融合进系统：

第四阶段完整回顾

附录：完整改造代码

main.py 关键变更：

```
class ChatBISystem:
    def run(
        self,
        user_question: str,
        use_few_shot: bool = True,
        use_rules: bool = True,
        use_guards: bool = True,
        use_indicator_knowledge: bool = True,
        use_schema_linking: bool = False,      # 第18课新增
        use_indicator_rag: bool = False,      # 第19课新增
```

```
) -> dict:
# ...
detected_indicators, indicator_block = self._resolve_indicator_context(
    user_question=user_question,
    use_indicator_knowledge=use_indicator_knowledge,
    use_indicator_rag=use_indicator_rag,
)

# Prompt 构造
system_msg, prompt = build_prompt(
    user_question,
    use_few_shot=use_few_shot,
    use_rules=use_rules,
    use_guards=use_guards,
    indicator_knowledge=indicator_block,
    use_schema_linking=use_schema_linking,
)
```

`run_stream` 流式方法也同步增加了 `use_schema_linking` 和 `use_indicator_rag` 参数，并复用同一个 `_resolve_indicator_context()`，避免同步链路和流式链路各自维护一份重复逻辑。

附件（飞书无下载权限）：第 19 课：指标知识库设计与 RAG 检索实战(1).pdf

附件（飞书无下载权限）：第 20 课：Schema Linking + RAG 全链路融合与系统升级(1).pdf